



Optimizing stock price forecasting: a hybrid approach using fuzziness and automated machine learning

Jan Timko ^a, Radwa El Shawi ^a, Stefania Tomasiello ^{b,a,*}

^a Institute of Computer Science, University of Tartu, Narva mnt 18, 51009, Tartu, Estonia

^b Department of Industrial Engineering, University of Salerno, Via Giovanni Paolo II, 132, 84084, Fisciano, SA, Italy

ARTICLE INFO

Keywords:

Fuzzy time series
Stock price prediction
Univariate time series
Automated machine learning
Fuzzy logic

ABSTRACT

Time series forecasting, particularly in the domain of stock prices, is a significant challenge but benefits from the availability of openly accessible data. Our work focuses mainly on (although not limited to) univariate time series forecasting of monthly or daily stock prices, predicting one step ahead. We developed an innovative pipeline that combines fuzzification with Automated Machine Learning, achieving improved forecasting performance. Unlike previous literature, we revise the binomial Fuzzy Time Series and machine learning algorithm, including a classification task (formally motivating it), and involving unused features of fuzzy sets. Thanks to the type of aggregation of the fuzzified data, the approach has the potential to preserve interpretability, unlike most machine learning based approaches. Using several financial datasets, in addition to preliminary experiments on chaotic time series, we found evidence of significantly improved performance in most cases. This study contributes to further understanding of the intersection between fuzzy logic and Automated Machine Learning, particularly in the context of time series forecasting, offering a promising direction for future research.

1. Introduction

Stock price forecasting is a major problem in time series forecasting. Predicting financial time series with accuracy can be difficult. This is especially true for stock prices.

Numerous fields, notably the stock market, have seen successful use of several approaches to forecasting models. Anyhow, when dealing with forecasting issues where time series data are imprecise and vague, classical time series methods are ineffective (Yolcu & Lam, 2017). The time series modelling approach using fuzzy set theory to model imprecision and vagueness is called fuzzy time series (FTS). FTS modelling approach appeared first in Song and Chissom (1993b).

Following the introduction of FTS models, subsequent studies concentrated on the FTS model's two main stages, i.e. fuzzification, resulting in fuzzy partitioning, and the creation of fuzzy relations. Fuzzy partitions rely on interval length (or support of fuzzy sets) and the type of adopted fuzzy set. The impact of interval length on forecast accuracy was investigated by Huarng (2001) and Huarng and Yu (2006). Evolutionary algorithms have also been used to optimize the interval length (Duru & Bulut, 2014; Egrioglu et al., 2010; Goyal et al., 2023; Zeng et al., 2019). Singh (2018) introduced a Self-Organizing Feature Map (SOFM) to partition the universe of discourse into intervals of different lengths.

The impact of different fuzzy sets in FTS modelling was studied in Bisht and Kumar (2023).

Regarding the identification of the fuzzy relations (FRs), several strategies have been proposed in the literature, starting with the matrix form Song and Chissom (1993a) to end up with neural networks (NNs) and their variants (PhamToan et al., 2024; Sadaei et al., 2019; Singh, 2017; Wu et al., 2021).

The hybridization of NNs with FTS represents an important advancement in the field of forecasting. By replacing the drawbacks of one technique with the advantages of another, it combines the best features of NN and FTS. This covers a number of NNs benefits, including parallel processing, managing big data, fast learning, etc. Fuzzy sets are utilized to handle imprecise or vague data, taking into account the uncertainty which normally affects time series. Using NNs for the identification of fuzzy relations, complex matrix operations, and complex fuzzy relation tables can be avoided.

Regarding the specific application of stock price forecasting, a hybrid model based on FTS and NNs was proposed by Yolcu and Alpaslan (2018).

The drawback with most existing techniques is that indices, as in the work by Sadaei et al. (2019), or clustering centres, as in the article by Yolcu and Alpaslan (2018), are considered instead of membership

* Corresponding author.

E-mail addresses: jan.timko@ut.ee (J. Timko), radwa.elshawi@ut.ee (R. El Shawi), stomasiello@unisa.it (S. Tomasiello).

values, and relationships are created on these, which may produce a loss of information. When it comes to merging fuzzy logic and machine learning (ML), it is critical to use the proper information. The challenge of selecting features from fuzzy sets was first discussed by Bonissone (1979) for linguistic approximation, using metrics like power, entropy, first moment and skewness.

On the other hand, as emphasized by Singh (2017), most hybrid approaches use only the previous one-day/year fuzzified value for forecasting. Such models are unable to fully capture the uncertainty residing in the events. Hence, fixing a proper number of lags is critical to the reliability of the model.

To address this, we propose herein an Automated Machine Learning (AutoML) procedure to determine the optimal number of lags for constructing a more accurate and reliable model. Moreover, AutoML is used to identify the optimal number and type of fuzzy sets, further enhancing the model's forecasting capability.

It must be recalled that the financial time series prediction is usually tackled as a regression problem. Park et al. (2019) tackled the stock market indices prediction as a classification problem, finding satisfactory results. Since the fuzzy discretization, i.e. the fuzzy partitioning, of the problem domain in classification problems seems to work efficiently (García et al., 2013), we redesign the stock price forecasting as a classification problem. We support this methodology with formal considerations as discussed in the related section. We leverage two AutoML solutions as classification models. First, we use the recent AutoML tool Tree-based Pipeline Optimization Tool (TPOT). Secondly, we apply a Multi-Layer Perceptron (MLP) with Bayesian search to optimize its architecture. It must be pointed out that the classification task is critical to the proposed AutoML procedure. In classical FTS, the next fuzzy set is predicted on the basis of fuzzy rules where the consequents are explicitly given. In our approach, the antecedents are given through fuzzification and aggregation, but the consequents are determined through a learning system. Setting the problem as a regression would mean predicting some values, strictly related to the features of a known membership function (MF), for instance, including triplets with centre, left and right spread. Anyway, the AutoML procedure does not work with a fixed MF type. The proposed procedure picks the best MF according to the optimization process. Thereby, the MF type is not known a priori in the forecasting process. As a consequence, any term of the fuzzy partition (represented through an MF) must be meant as a class, bringing in itself all the features of the fuzzy set.

From a numerical perspective, we performed several experiments to compare our approach against state-of-the-art methods, which involve or do not involve fuzzy set theory. As a first analysis, we considered well-known chaotic time series, such as those from the Hénon map and the Lorenz system. The related results were compared against those from the recent relevant literature (Jayawardena et al., 2006; Su & Wang, 2025), where a Radial Basis Function (RBF) model and the Kernel Broad Learning Cauchy Conjugate Gradient (KBLCCG) algorithm were respectively used. The reason for starting with such time series is that the dynamics involved is representative of real-world scenarios. In particular, the Hénon map has become a standard benchmark in nonlinear time series analysis (Kantz & Schreiber, 2003) and has been used to simulate or model phenomena in fields such as neuroscience (Skarda & Freeman, 1987) and control engineering (Ott et al., 1990). On the other hand, the Lorenz chaotic system is traditionally related to hydrodynamic flow, with an impact on weather and climate systems (Lorenz, 1963).

After exploring chaotic time series, we considered several financial datasets, such as Shanghai Monthly (Wang et al., 2011), Shanghai Daily (Wang et al., 2021), TAIEX (Wu et al., 2021), a smaller TAIEX dataset (Kumar & Pant, 2025) and seven stocks from the S&P 500 index (Gkonis & Tsakalos, 2025), comparing, in the order, against a standard back-propagation Neural Network (BPNN), a Reservoir Computing (RC) model, FTS with Affinity Propagation clustering, and a Support Vector Regression (FTS-AP-SVR), Probabilistic Intuitionistic Fuzzy Sets

with Adaptive Radius Clustering and Markov chain (PIFS-ARC-Markov), and Golden Jackal Optimizer with Gated Recurrent Unit or Long Short-Term Memory (GJO-GRU|LSTM). Finally, we also considered the multivariate case, with the Turkish lira exchange rates datasets using the dollar, euro and pound datasets as factor variables, comparing against the recent Multivariate Picture Fuzzy Time Series with Pi-Sigma Artificial Neural Networks (MPFTS-PSANN) (Bas et al., 2023), a kind of fuzzy inference system (TSFIS) (Yolcu & Alpaslan, 2018), a kind of fuzzified neural network (FF-ANN-PSO) (Egrioglu et al., 2016).

To summarize, the main contributions of this work are as follows.

- We use AutoML for the first time (to the best of our knowledge) to optimize fuzzy partition parameters, such as the number of fuzzy sets and MFs, ensuring effective model tuning. We frame the fuzzy time series forecasting with a classification task, formally motivating it. Specifically, we predict fuzzy partition terms based on scaled period-over-period percentage differences. AutoML optimization helps us achieve strong results in stock index prediction.
- Our feature engineering approach captures granular details from time series data, not only extracting fuzzy partition terms, membership values, and information about the position of the data point within the fuzzy set but also dealing with their lagged values. In fact, the number of lags is also optimized, controlling the number of features used and tailoring the model specifically to our dataset, thereby enhancing its predictive performance.
- Since we do not use ML at the aggregation stage, but we rely on a classical logical connective for the aggregation of fuzzified data, the approach has the potential to preserve interpretability, which, to the best of our knowledge, seems not to be addressed or discussed in the most recent literature merging FTS and ML.

The paper is structured as follows. The next section is devoted to the related work. Section 3 illustrates the background, recalling some preliminaries. Section 4 describes the proposed approach. In Section 5, the numerical experiments were discussed. Finally, some conclusions are drawn in Section 6.

2. Related work

In this section, the relevant literature on FTS and AutoML is presented.

2.1. Early contributions

Stock time series prediction problems can be grouped into two classes, i.e. value forecasting and trend forecasting (Tealab et al., 2017).

In the first case, the future values of a time series are estimated, while qualitative behaviour is predicted in the second case. Value forecasting is a regression task, which has been tackled using several techniques (DeGooijer & Hyndman, 2006). The most popular method is the Auto-Regressive Integrated Moving Average (ARIMA) model (DeGooijer & Hyndman, 2006), which generates a value prediction within a confidence margin based on its past observations. Several variants of the original ARIMA model have been developed (Box, 2015). Anyway, the ARIMA model does not capture the non-linear patterns properly. Besides, in order to improve forecasting, the traditional time series approach needs additional historical data, and that data also needs to have a normal distribution. Additionally, linguistic expressions are frequently employed to describe daily observations. Thus, Song and Chissom (1993b) established the foundation for the FTS hypothesis, which has seen widespread use in the past few years. An FTS model was presented in Song and Chissom (1993a) to forecast UA's enrollments. This model described a systematic process utilizing max-min operations and suggested a first-order, time-invariant method.

Over the last few decades, the notion of FTS has undergone significant evolution. In the work by Chen (1996), good results were obtained

using arithmetic operations rather than the more common max-min procedures used by Song and Chissom (1993a). Subsequently, Hwang et al. (1998) presented a time-variant FTS method for forecasting demonstrating their methodology with historical enrollment data. In order to anticipate enrollments for the upcoming year, this method uses systematic computations to correlate deviations from the previous year, recognizing that past trends have an impact on current trends. Chen (2002) presented a sophisticated FTS model, building on previous developments. The objectives of this higher-order model were to increase forecasting accuracy and decrease uncertainty. A simple computational method for FTS forecasting with different parameters was proposed by Singh (2007). This technique was tried to predict wheat production. The FTS approach by Cheng et al. (2008) took into account numerous attributes and fuzzy grouping. This technique makes it possible to segment datasets objectively and handle various attributes effectively.

2.2. Heuristic and metaheuristic-based FTS

Bai et al. (2011) presented a heuristic time-invariant forecasting model for FTS. The model trains the trend predictor and produces forecasts by utilizing prediction accuracy. In an effort to increase prediction accuracy, Kuo et al. (2009) presented an improved forecasting model which combines FTS and particle swarm optimization (PSO). An FTS forecasting strategy based on the combination of optimal interval partitions and PSO was introduced by Chen et al. (2019). Another forecasting model using PSO, but this time combined with neutrosophic set theory, was discussed by Singh (2020). This new method uses neutrosophic data to express time series data. Wong et al. (2010) created an adaptive time-variant forecasting model using FTS as a framework. During the training phase, this model automatically modifies the window size to maximize the prediction accuracy.

A high-order FTS model with interval length optimization using the Harmony Search Intelligence algorithm was proposed by Jiang et al. (2017). A moving average method was developed specifically for FTS data by Gautam and Abhishekh (2019). Its purpose was to reduce data volatility and improve forecast accuracy.

2.3. Machine-learning enhanced FTS

In their study of FTS forecasting, Panigrahi and Behera (2020) focused on fuzzy logic relationships (FLRs) modelling and accurate interval length determination. To represent FLRs, they combined machine learning techniques such as deep belief networks, long short-term memory, and support vector machines.

Yolcu and Alpaslan (2018) used Fuzzy C-Means (FCM) clustering for the fuzzification and single multiplicative neuron model (SMNM) to determine the fuzzy relations. The training process of this computational scheme was based on PSO. Egrioglu et al. (2016) introduced a FTS forecasting method that combines fuzzy logic with an NN optimized using PSO (denoted as FF-NN-PSO). The method first applies FCM clustering to the data, and then trains a multiplicative neural network using PSO. More recently, FCM was used jointly with type-2 fuzzy sets (Satriani et al., 2025). The authors used only one dataset and did not discuss any comparison against existing approaches, making their contribution debatable, considering the complication induced by high-order fuzzy sets.

Wu et al. (2021) employed affinity propagation (AP) clustering for fuzzification and SVR for the fuzzy relations. PhamToan et al. (2024) proposed a hybrid FTS model consisting of self-updating clustering and bidirectional LSTM.

Sadaei et al. (2019) developed the idea of converting FTS to the format of images in the input layer of a Convolutional Neural Network (CNN).

Zhan et al. (2024) used a convolutional layer in their scheme to reimplement FTS, using the difference between each time node in the time series to ensure stationarity. Considering only the extremal membership

values to a fuzzy set, i.e. $\{0, 1\}$, they used a formula inherited from binary encoding to fix the number of fuzzy sets in the partition, i.e. for a time series with N elements, $\lceil \log_2(N) \rceil$. They did not rely on membership functions but only intervals, as derived from the support of the fuzzy sets. In fact, the convolution layer in their scheme was aimed at replacing the membership functions to make the inference independent from fuzzy variables. It is clear that this poses problems with interpretability, which should be the main reason for using fuzzy logic.

Kocak et al. (2020) used a type of NN to enhance FTS. They used lagged fuzzy variables. Inputs to the NN were the terms of the partition corresponding to the maximum membership values of the lagged variables. The NN worked as a regressor to predict the one-step-ahead term. The input values and targets were integers denoting the terms of the fuzzy partition. Anyhow, it is widely accepted that when the target values come from a discrete finite set of integers, regression can be efficiently replaced by classification (Bishop, 1995).

In this respect, it is useful to point out that in the present work, to ensure stationarity, we consider percentage differences between successive data points in the time series, moving to a dimensionless domain. We consider the influence of the type of membership function, the size of the fuzzy partition, and the number of lags, as they are critical to accuracy and interpretability, using AutoML to find the optimal combination. We recast the FTS forecasting problem as a classification problem to predict the term of the fuzzy partition, which is linked to the one-step ahead value in forecasting.

As pointed out by Zhan et al. (2024), in the usual FTS forecasting, the input is the data point and the interval to which that point belongs, and NNs can contribute to improving the fuzzy inference, even though FTS based on NNs are not able to tackle the improper process once the fuzzy sets are fixed. In this sense, AutoML techniques are more flexible, providing the optimal computing scheme.

2.4. Multivariate and non-standard FTS

Liu (2023) proposed a multivariate FTS model to improve stock price prediction by integrating both quantitative and qualitative factors into the prediction process. The Principal Component Analysis (PCA) was applied to identify key influencing variables, thereby reducing the dimensionality of the input space while retaining relevant information. A clustering algorithm was then introduced to construct fuzzy sets over the derived universe of discourse, followed by the formation of a first-order Markov transition matrix to model the evolution of fuzzy relationships over time. Empirical evaluation was conducted on two datasets, but the experimental part of this work shows its main limitation, i.e. the very small size of the dataset, which poses doubts on the application of the approach in a real-world scenario.

Bas et al. (2023) explored time series forecasting through the lens of picture fuzzy time series (PFTS), a generalization of traditional fuzzy sets that incorporates degrees of positive, neutral, and negative membership, along with a refusal degree. They proposed a Multivariate Picture Fuzzy Time Series (MPFTS) model designed to capture high-order and multivariate dependencies within time series data. To manage the non-linearity and complexity inherent in such data, the authors integrated a Pi-Sigma Artificial Neural Network (PS-ANN) trained via Particle Swarm Optimization (PSO). The strength of their framework lies in its capacity to model uncertainty in a nuanced way and in its flexibility for integrating external covariates to improve forecasting accuracy. Anyhow, optimal parameter tuning is a critical aspect that the authors did not discuss and that should be addressed.

In the present work, we will address this issue by using AutoML techniques. For the sake of completeness, it must be mentioned that (Kumar & Pant, 2025) proposed a hybrid FTS forecasting method (PIFS-ARC-Markov), combining probabilistic intuitionistic fuzzy sets, adaptive radius clustering, and a Markov chain to improve uncertainty handling, partitioning, and FLR weighting in FTS. The authors used small datasets, making the evaluation of the approach in real-world scenarios difficult.

2.5. AutoML and fuzzy logic

There are very few works where AutoML and fuzzy logic appear, although in different applications. The work by Batrouni (2020) demonstrated that combining fuzzy logic with AutoML can yield an improved intelligent tiering system for storage optimization. The suggested architecture comprises a fuzzy inference system to choose the eligible tiers based on certain business criteria, along with an AutoML component like Auto-SKLearn that recommends a classification algorithm from a pre-established list. The research conducted by Tsiakmaki, Kostopoulos, Kotsiantis, Ragos, (2021) presents a fuzzy-based active learning approach for forecasting students' academic performance, utilizing AutoML methodologies. The authors examined the first six auto-configured representative fuzzy classifiers, conducting numerous experiments to choose the most promising candidates. Secondly, four active learning models were developed, and AutoML was utilized to automate the selection of fuzzy machine learning models, based on the three dominant classifiers from the earlier comparative study. In Nayak et al. (2021), AutoML and fuzzy C-means are examined regarding their application in the analysis of biomedical data. The latter area is drawing increasing attention because of the intricate datasets. For this purpose, the authors deliberated on a model grounded in FCM that utilizes meta-heuristic optimization. Amini et al. (2024), in order to lower the computational burden required by the multicriteria decision-making approach Fuzzy PROMETHEE (F-PROMETHEE), proposed a surrogate model. The procedure was built on first generating a small design of experiments (DoEs) from the actual model, and then approximating the quantity of interest from the meta-model (that could be defined as model of the model), which was calibrated to the initial DoEs. The meta-model was obtained as the best fit (non-fuzzy) ML technique through the AutoML procedure based on PyCaret. Wang et al. (2024) proposed a Lightweight Spatio-Temporal Decoupling Transformer based on AutoML. The authors, first applied a kind of positional encoding, which is improperly called fuzzy, as it simply includes stochasticity, then they used Depthwise convolution to extract temporal information and variable structural information independently from the data. Subsequently, Pointwise convolution was employed to combine the temporal and structural information. Finally, linear self-attention was used to model the fusion features on a global scale, allowing for adaptive enhancement of critical information within those features. The AutoML procedure was meant for the Transformer's hyperparameters tuning. They applied their approach to classification problems. They improperly wrote time series classification to mean a classification task with data varying over time.

In the present work, we propose to use AutoML to solve the tedious issue of a proper choice in fuzzy partitioning with respect to the specific forecasting task herein considered, i.e. number and type of fuzzy sets, in addition to a possible number of lags.

3. Background

In order to support the discussion on the proposed approach, some basic notions on fuzzy sets, FTS, and AutoML are herein recalled.

3.1. Fuzzy sets and fuzzy time series

Definition 1. A fuzzy set A can be regarded as a set of ordered pairs over a universe (of discourse) U

$$A = \{(u, \mu_A(u)) | u \in U\}, \quad (1)$$

where

$$\mu_A : U \rightarrow [0, 1] \quad (2)$$

is the membership function of A and $\mu_A(u)$ is the degree of membership of u in A . If $\max_{u \in U} \mu_A(u) = 1$, then the fuzzy set is called *normal*.

Notice that a normal and convex fuzzy set represents a *fuzzy number*.

The membership function (MF) uniquely identifies a fuzzy set. A typical MF is the *triangular function*:

$$\mu_A = \begin{cases} \frac{x-x_{j-1}}{x_j-x_{j-1}}, & x \in (x_{j-1}, x_j] \\ \frac{x_{j+1}-x}{x_{j+1}-x_j}, & x \in (x_j, x_{j+1}) \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

Other well-known MFs are the *cosine function*:

$$\mu_A = \begin{cases} \frac{1}{2} \cos\left(\frac{\pi(x-x_j)}{x_j-x_{j-1}}\right) + 1, & x \in (x_{j-1}, x_j] \\ \frac{1}{2} \cos\left(\frac{\pi(x-x_j)}{x_{j+1}-x_j}\right) + 1, & x \in (x_j, x_{j+1}) \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

and the *Gaussian function*:

$$\mu_A = \begin{cases} \exp\left(-\frac{(x-x_j)^2}{2\sigma^2}\right), & x \in (x_{j-1}, x_{j+1}) \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

Definition 2. A *fuzzy partition* of the interval $X = [x_1, x_m]$ is the sequence $\{\mu_{A_i}(x)\}_{i=1, \dots, m}$ of normal continuous fuzzy sets A_i , satisfying the Ruspini's condition $\sum_{i=1}^m \mu_{A_i} = 1, \quad \forall x \in X$.

A fuzzy partition is called *uniform* when the nodes are equidistant. In general, $h = \max |x_{i+1} - x_i|$ is the norm of the partition. If the partition is uniform, then $h = (x_m - x_1)/m$ and $x_j = x_1 + jh$.

The logical connectives to aggregate membership values are the t-norm, generalizing the AND operator, and the t-conorm, generalizing the OR operator. There are several types of t-norm and t-conorm. In our methodology, the type of t-conorm we will use is the maximum.

Definition 3. Let $T = \{x(t_1), x(t_2), \dots, x(t_N)\}$ be a time series of N elements. The *fuzzy time series* over T is the sequence of membership degrees of $x(t_i)$ to the given fuzzy set A , $\bar{T} = \{\mu_A(x(t_1)), \dots, \mu_A(x(t_N))\}$.

In Definition 3 and what follows, it is assumed a time-invariant fuzzy time series, i.e. at any two successive time points t_1 and t_2 , we have the same fuzzy sets.

Following the methodology described by Song and Chissom (1993a,b), the fuzzy time series (FTS) forecasting model relies on the following steps:

- defining the universe of discourse (UOD),
- fuzzification of historical data with proper fuzzy partitioning of the UOD,
- defining some fuzzy relations to compute the results,
- converting the results into crisp numbers.

The differences between this standard approach and the proposed approach are highlighted in Section 4.

3.2. Automated machine learning

Automated Machine Learning (AutoML) has become an essential paradigm in modern machine learning, aiming to automate the end-to-end process of model development. This includes data preprocessing, feature selection, model selection, and hyperparameter optimization (Abd Elrahman et al., 2020; El Shawi et al., 2025; Eldeeb et al., 2022). By reducing reliance on manual intervention and specialized expertise, AutoML facilitates broader accessibility to machine learning technologies while enhancing efficiency, reproducibility, and methodological consistency (Hutter et al., 2019). AutoML frameworks have played a central role in operationalizing this automation for both academic research and practical applications. These tools typically employ algorithmic strategies such as random search, Bayesian optimization, evolutionary algorithms, and meta-learning to explore and optimize

over complex model and preprocessing pipelines (Feurer et al., 2015; Hutter et al., 2011).

One of the most widely adopted frameworks is Auto-sklearn (Feurer et al., 2015), which extends the scikit-learn ecosystem by integrating Bayesian optimization and meta-learning. It automates model selection and hyperparameter tuning while also supporting automated ensemble construction, thereby enhancing predictive performance and robustness (Feurer et al., 2015). MLJar AutoML provides a fully automated framework for both classification and regression tasks, supporting a range of algorithms including tree-based models and linear classifiers. It emphasizes interpretability and transparency through structured model reports and diagnostics. H2O AutoML (LeDell & Poirier, 2020), although also available in enterprise versions, includes a powerful open-source module that supports automatic training and ranking of multiple models using gradient boosting, random forests, generalized linear models, and stacked ensembles. It is designed to scale efficiently across large datasets and computational resources.

In this study, we utilized the AutoML tool Tree-based Pipeline Optimization Tool (TPOT), introduced by Olson et al. (2016). TPOT is an open-source AutoML framework designed to construct classification and regression pipelines through the use of genetic programming. Machine learning pipelines within TPOT are represented as computational graphs, where distinct branches correspond to alternative preprocessing and modeling strategies. The framework employs a multi-objective optimization algorithm that simultaneously seeks to maximize predictive performance and minimize pipeline complexity. This trade-off is intended to reduce the risk of overfitting, which can arise due to the extensive search space inherent in automated pipeline generation. TPOT has been proven to outperform baseline models in just minutes of training (Eldeeb et al., 2024).

The search space of TPOT includes classifiers, feature preprocessors, and feature selectors, primarily from the scikit-learn library, as well as one classifier from the XGBoost Python library. The classification operators available in TPOT include, among others, Decision Tree, Random Forest, XGBoost, KNN, and Logistic Regression. Feature preprocessing operators in TPOT include scaling methods such as StandardScaler, RobustScaler, and MinMaxScaler, as well as Principal Component Analysis (PCA). Examples of feature selection operators are RFE, Variance Threshold, and SelectKBest (Olson et al., 2016). By looking at the classifiers in the search space of TPOT, it is worth mentioning that the adopted loss functions are mostly classification-calibrated, ensuring the optimal value of the expectation over the loss (Bartlett et al., 2006).

4. The proposed approach

The theory of fuzzy sets allows to analyze quantitative factors as well as consider the qualitative information, which is described by linguistic values. It is well-known that qualitative forecasting relies on expert judgment and is used when historical data is scarce or unavailable. It is helpful in uncertain situations, but can be biased due to factors like anchoring and selective perception. These forecasts can guide decision-making, though they lack the precision of quantitative methods. Quantitative forecasting, in contrast, uses historical data to identify patterns and predict future outcomes through statistical models. Over the past two decades, fuzzy time series models have gained popularity for handling vague or linguistic data in uncertain environments. These models depend heavily on defining accurate fuzzy logical relationships. While fuzzy logic handles ambiguity well, it lacks learning capabilities. Neural networks, and learning systems in general, can learn from data but struggle with vague or probabilistic inputs. Combining both helps balance these strengths.

In this work, we focus on automating the fuzzification process in ML by automating key steps such as the selection of MFs, the number of elements of the fuzzy partitions, and the number of lags used in the time series. Our approach utilizes AutoML techniques to optimize these parameters, addressing the challenges that were traditionally handled

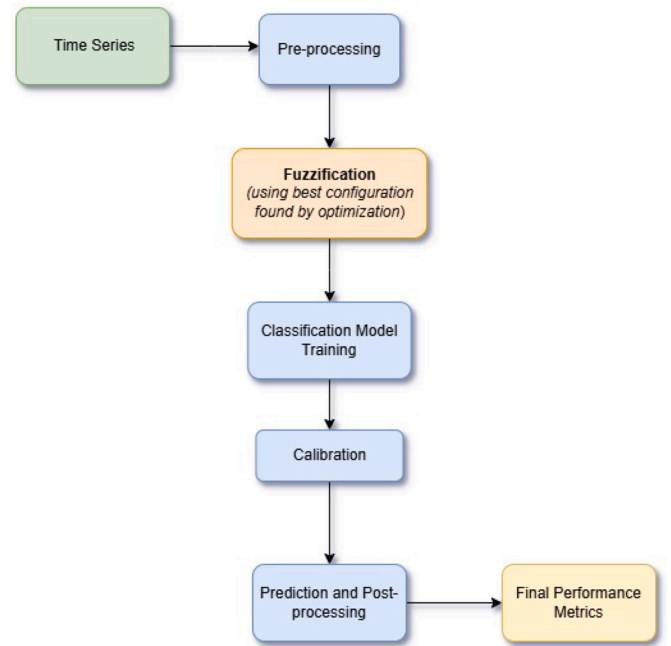


Fig. 1. Simplified schema of the pipeline.

manually by domain experts. By automating these critical steps, we aim to enhance the scalability, flexibility, and effectiveness of applying fuzzy logic in ML tasks. The proposed pipeline consists of the main stages shown in Fig. 1 and described in the next subsections.

4.1. Data preprocessing

The first step in the pipeline is the preprocessing of the time series data. Initially, the data is scaled to a 0–1 range using a standard scaler fitted to the training data. This scaling is then applied to both the validation and test datasets to ensure consistency across the dataset. Following the scaling, the series is transformed using percentage differences to ensure stationarity, also in view of time-invariant fuzzy time series. The transformation is defined as:

$$\bar{x}(t) = \frac{x(t) - x(t-1)}{x(t-1)} \times 100 \quad (6)$$

where $x(t)$ represents the original value at time t , and $x(t-1)$ is the value at the previous time step. This transformation ensures that the data is stationary and removes any trends that might affect model performance.

4.2. Fuzzification of the data

Once the search space has been defined and the optimal hyperparameters identified, the fuzzification process is initiated. This process entails applying fuzzy partitioning to the dataset. The MF is selected according to the configuration determined during the optimization phase. For each data point $\bar{x}(t)$, the membership values $\mu_k(\bar{x}(t))$ are computed (with k varying from 1 up to the number of fuzzy sets in the partition). A max-type t-conorm is then applied, which could be roughly interpreted as if the data point was assigned to the fuzzy set with the highest membership value.

In the case of the Gaussian MF, an additional hyperparameter σ_h is introduced to compute the Gaussian parameter σ , where $\sigma = \frac{h}{\sigma_h}$, with h representing the norm of the fuzzy partition. This parameterization ensures that the granularity of the fuzzy partition is adapted to the underlying data distribution.

For each data point $\bar{x}(t)$, the membership values are calculated throughout the partition. Then, the max-type t-conorm is applied. The

membership value itself does not provide sufficient information to determine the location of a data point $\bar{x}(t)$ within the fuzzy set. Hence, we introduce a boolean variable, denoted as b , which indicates whether a point $\bar{x}(t)$ belongs to the left-hand ($b = 1$) or right-hand side of the fuzzy set ($b = 0$) with respect to the center of the fuzzy set, for a given membership value.

4.3. Search space for model training

The search space defines the range of possible values for the hyperparameters controlling the model. These hyperparameters include the number of fuzzy sets, the number of lags, and the MF (triangular, cosine, or Gaussian functions). For the Gaussian MF, we introduce an additional conditional hyperparameter, σ_h , which is used to calculate the Gaussian parameter σ . The search space for the key hyperparameters that control the model is detailed in Section 5, which is devoted to the numerical experiments.

The search space is optimized using Bayesian optimization to identify the optimal values of the hyperparameters. The goal of the optimization process is to select the number of fuzzy sets, the type of MF, and the number of lagged values that best suit the data for accurate forecasting.

4.4. Classification, calibration and post-processing

We start off this section with a proposition to formally illustrate the reason behind the classification problem.

Proposition 1. Let $\mathcal{L}$ be a learning system with loss function $\phi(y) : \mathbb{R} \rightarrow \mathbb{R}$. Let $\phi(y)$ be convex on the interval $[-\beta, \beta]$, with $\beta \in \mathbb{R}_+$, and let $\bar{y} \in \mathbb{N}$. Assume there exist a metric d and a constant L_β such that $L_\beta d(y, \bar{y}) < \epsilon$, with $0 < \epsilon < 1$. Then $|\phi(\bar{y}) - \phi(y)| < \epsilon$.

Proof. Straightforwardly, since d is a metric and ϕ is convex, then ϕ necessarily satisfies a Lipschitz condition and L_β is the Lipschitz constant (Rockafellar, 1997). $\square$

Proposition 1 can be read in the following way: let us suppose there are two learning systems using the same loss function, with only one able to predict integers; under a certain condition, the difference between the two systems is negligible.

Notice that Proposition 1 is not trivially satisfied. First, the distance of a regression-type prediction from an integer may be greater than 1. Secondly, the Lipschitz constant is not necessarily small. In fact, if one considers the following convex loss functions and corresponding Lipschitz constants (Bartlett et al., 2006)

- exponential, $\phi(y) = e^{-y}$, $L_\beta = e^\beta$,
- quadratic, $\phi(y) = (1 - y)^2$, $L_\beta = 2(\beta + 1)$,

it is possible to see that L_β depends on β , which may not be small.

This supports the considerations behind the classification when targets are from a finite set of integers.

With such a premise, we illustrate in the following the next stage of the proposed methodology. The fuzzified data is aggregated by means of a max-type t-conorm. The fuzzy set associated with the maximum membership value will fall in a certain class since each term of the fuzzy partition will denote a class C .

The number of lagged values n is selected through an AutoML approach.

For each lag $i \in \{1, \dots, n\}$, the following features are computed:

- $C_i(t)$: the i -th lag of the class, calculated at time t ;
- $\bar{\mu}_i(t)$: the i -th lag of the membership value, calculated at time t ;
- $b_i(t)$: the i -th lag indicating whether the point belongs to the left-hand side of the fuzzy set, calculated at time t .

In the classification model, the input variables are all the lagged features for $i \in \{1, \dots, n\}$, where n is the hyperparameter determining the maximum number of lags to include. Thus, the total number of input

features in the model is $3n$, as each lag contributes three features. The target variable for the model is the class label at the current time period.

The best configuration identified through the optimization process is further calibrated using the validation dataset. Calibration adjusts the model's predicted probabilities to ensure they are properly scaled, providing a more accurate representation of the true likelihood of the target class. A common approach is logistic calibration, i.e. Platt scaling (Silva Filho et al., 2023).

Once the optimal fuzzy set configuration has been determined and the classifier is calibrated, the final output is obtained. The model utilizes the best configuration to provide the class associated with the one-step-ahead value. The original percentage difference is then recalculated by means of the center of the fuzzy set corresponding to the predicted class and the previously obtained value (in the original scale) to produce the final prediction. This formally turns into (7)

$$x(t+1) = x(t) \times \left(1 + \frac{\hat{x}(t+1)}{100}\right) \quad (7)$$

where $\hat{x}$ is the predicted percentage difference and $x(t)$ is the previously obtained value (in the original scale).

Thanks to the fuzzification and logical connective, it is possible to extract some rules. For instance, if the fuzzy partition consists of $m = 2$ fuzzy sets, representing the terms {LOW, HIGH}, and the number of lags is $n = 2$, we would have rules such as

- IF $x_2(t)$ IS LOW OR $x_1(t)$ IS LOW THEN $x_2(t+1)$ IS LOW,
- IF $x_2(t)$ IS HIGH OR $x_1(t)$ IS HIGH THEN $x_2(t+1)$ IS HIGH.

5. Numerical experiments

This section details the datasets, experimental settings, evaluation methods, optimization processes, and results for both simulated and real-world datasets used in our study. It must be pointed out that even though the proposed approach has potential for qualitative forecasting, it has been tested for quantitative analysis and compared in this sense against state-of-the-art methods. The experiments were run on a machine with an Intel Core i5-1135G7 CPU (4 cores) and 16 GB RAM. The code was implemented in Python 3.11.5 using libraries including scikit-learn 1.5.3, XGBoost 2.1.2, TPOT 0.12.2, Optuna 4.0.0, NumPy 2.1.2, and Pandas 2.2.3.

5.1. Experimental data and settings

Our experiments were conducted mainly on univariate time series datasets with daily and monthly frequencies, but we also performed experiments on multivariate time series. The forecasting horizon was one step ahead, and we followed the same training and testing ranges as in prior studies to ensure comparability with baseline methods.

All the datasets used in our numerical experiments and the corresponding references from which the baselines are obtained are tabulated in Table 1. Further details are presented in separate tables in the following.

The experiments started with two well-known chaotic time series, namely the Hénon map (Jayawardena et al., 2006) and Lorenz chaotic series (Su & Wang, 2025), as they are commonly used benchmarks for testing forecasting models. Table 2 summarizes the number of samples and train/test splits for those datasets.

Subsequently, we evaluated our method on financial time series datasets representing stock market indices: Shanghai Monthly (Wang et al., 2011), Shanghai Daily (Wang et al., 2021), and TAIEX (Wu et al., 2021). These datasets provide a range of market conditions for evaluating model performance on real-world data. Additionally, we performed experiments with the S&P 500 index datasets (Gkonis & Tsakalos, 2025) and the exchange rates (Bas et al., 2023).

The yearly TAIEX dataset uses a varying number of samples per year; therefore, the exact sizes are provided separately. Table 3 presents the yearly data splits and corresponding sample size.

Table 1
Summary of datasets and baseline references.

Dataset	Number of samples	Test size	Reference
Hénon Map	10,000	5000	(Jayawardena et al., 2006)
Lorenz Series	4000	800	(Su & Wang, 2025)
Shanghai Monthly	204	204	(Wang et al., 2011)
Shanghai Daily	2188	2188	(Wang et al., 2021)
TAIEX (2010-2019)	220-230 per year	41-44	(Wu et al., 2021)
TAIEX (Nov-Dec 2014)	45	11	(Kumar & Pant, 2025)
S&P 500 Selected Stocks	1258 per stock	251	(Gkonis & Tsakalos, 2025)
TRY Exchange Rates (2015-2018)	260-261 per year/pair	40	(Bas et al., 2023)

Table 2
Chaotic time series.

Dataset	Total Samples	Test Samples
Hénon Map	10,000	5000
Lorenz	4000	800

A standard 60/20/20 split for training, validation, and testing is used for numerical experiments, unless specified otherwise. Each class is denoted with a number, in the discrete range from 1 to the number of fuzzy sets forming the fuzzy partition.

5.2. Evaluation metrics and statistical significance

The evaluation metrics were selected to align with those used in baseline studies. In the following formulas, N denotes the total number of samples, $\hat{y}_i$ is the actual value and y_i is the predicted value for the i -th sample. Hence, the adopted metrics are:

- Mean Absolute Error (MAE), which measures the average magnitude of forecast errors and it is computed as

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|, \quad (8)$$

- Mean Squared Error (MSE), which squares error magnitudes, emphasizing larger deviations, and it is given by

$$MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad (9)$$

- Root Mean Square Error (RMSE), which takes the square root of MSE, i.e.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}. \quad (10)$$

Table 3
TAIEX dataset sizes.

Year	Total samples	Test samples
2010	230	44
2011	227	43
2012	227	41
2013	224	42
2014	228	42
2015	224	43
2016	222	43
2017	223	42
2018	225	41
2019	220	41

Table 4
Search space for Hénon dataset optimization.

Hyperparameter	Range of values
Number of Lags	2 to 5
Number of Fuzzy Sets	2 to 29
Fuzzy Partition Functions	Cosine, Triangle, Gaussian
Parameter σ_h (for Gaussian)	Grid: {0.1, 0.5, 1, 2, 3, 4} Bayesian: Log-uniform [0.1, 4]

Anyhow, since in our methodology, a classification task is involved, we also consider the classification accuracy:

$$Accuracy = \frac{|T_N| + |T_P|}{|T_N| + |T_P| + |F_N| + |F_P|}, \quad (11)$$

where $|T_P|$ and $|T_N|$ stand for true positive and true negative, $|F_P|$ and $|F_N|$ denotes false positive and false negative, respectively.

To assess the statistical significance of our results, we calculated 95 % confidence intervals (CIs) for each metric. The intervals were derived from the error distributions and compared against baseline values. Improvement was considered significant if the baseline metric fell outside our CI bounds.

5.3. Optimization and calibration

We used Grid Search and Bayesian Optimization to optimize hyperparameters, varying on the basis of the dataset.

For the Hénon dataset, both Grid Search and Bayesian Optimization were applied, exploring the hyperparameter space described in Table 4.

We performed 320 trials for Bayesian Optimization, matching the number of configurations evaluated via Grid Search.

Initially, we evaluated several optimization techniques in parallel. However, due to its superior computational efficiency and ability to rapidly converge on high-performing configurations, we subsequently continued with Bayesian optimization exclusively.

In fact for the financial datasets (Shanghai Monthly, Shanghai Daily, TAIEX, S&P 500 stocks, and Exchange Rates) as well as the Lorenz chaotic series, we employed Bayesian Optimization exclusively. The search space was defined as detailed in Table 5.

The number of trials (315) was aligned with the computational budget for comparability across datasets. Initially, we used a much wider range of hyperparameters. After several experiments and by analyzing the distribution of results and selected configurations, we narrowed the search space. This was done carefully to ensure that the best configurations were not located near the boundaries, which would otherwise underutilize the AutoML capabilities.

In this study, the model was calibrated using CalibratedClassifierCV from scikit-learn. We configured it to perform five-fold cross-validation and applied the logistic calibration method.

5.4. Numerical results

5.4.1. Hénon map dataset

The best configurations found for the Hénon dataset are presented in Table 6. We used both Grid Search and Bayesian Search to optimize the fuzzy partition, achieving better performance with Grid Search. Both methods are fairly consistent in selecting the top configuration as the

Table 5
Search space for financial datasets optimization.

Hyperparameter	Range of values
Number of Fuzzy Sets	4 to 40
Number of Lags	1 to 10
Fuzzy Partition Functions	Cosine, Triangle, Gaussian
Parameter σ_h (for Gaussian)	Log-uniform [0.1, 10]

Table 6
Best configurations for Hénon dataset.

Dataset	Validation set	Optimization method	Best configuration
Hénon	3x1000 samples	Grid Search	26 fuzzy sets, 4 lags, Gaussian ($\sigma_h = 3.0$)
Hénon	1000 samples	Bayesian Search	24 fuzzy sets, 5 lags, Gaussian ($\sigma_h = 3.0$)

difference is only very small: Grid Search selected 26 fuzzy sets and 4 lags, while Bayesian Search identified the best configuration as 24 fuzzy sets and 5 lags. In both cases, Gaussian membership functions with $\sigma_h = 3.0$ were selected, indicating their suitability for the Hénon dataset.

For the sake of completeness, we present the intermediate results of the calibrated classification model in Table 7. The predicted classes are then associated with the corresponding center points of the fuzzy sets and undergo post-processing before the final evaluation. For the Hénon case, we achieved nearly 96 % accuracy for both the Bayes and Grid search methods, which is a good result since there are 24-26 classes. We found similar accuracy for the cases discussed in the next sections, and it is not reported for the sake of brevity.

The evaluation of these configurations on the test set did not achieve significant improvements over the baseline. As shown in Table 8, the Grid Search configuration achieved an MSE of 1.328×10^{-3} , which is lower compared to the Bayesian Search with an MSE of 1.553×10^{-3} . However, the overlapping confidence intervals suggest comparable performance.

We used the same configuration space and number of evaluations for both Grid Search and Bayesian Search. As a result, the total pipeline execution time, dominated by model training and evaluation, remained approximately the same.

Bayesian Search introduces additional computational overhead due to the need to fit a surrogate model, predict the performance of new candidates, and select the most promising one. In the initial phase, for the first N_{startup} trials (default is 10), random sampling is used since there are not enough observations to reliably fit the surrogate model.

The additional computational cost of Bayesian Search can be approximated as:

$$\text{Additional Cost}_{\text{Bayes}} = (N_{\text{trials}} - N_{\text{startup}}) \times (t_{\text{fit}} + t_{\text{predict}} + t_{\text{select}}).$$

However, since the surrogate model is relatively simple and fast to compute compared to the full pipeline, the difference in computational cost between Bayesian and Grid Search is minimal.

Both Grid and Bayesian Search show significant improvements over the older RBF model (Jayawardena et al., 2006). These results suggest that combining fuzzification with AutoML techniques can be an effective

Table 7
Accuracy results for the Hénon dataset (classification models).

Dataset	Number of classes	Accuracy
Hénon (Grid Search)	26	95.6 %
Hénon (Bayes Search)	24	95.7 %

Table 8
Test results for Hénon dataset with 95 % confidence intervals.

Dataset	Metric	Result (XGBoost)	95 % CI lower	95 % CI upper	RBF (Jayawardena et al., 2006)
Hénon (Grid Search)	MSE	1.328e-03	8.10e-04	1.845e-03	2.080e-03
Hénon (Bayes Search)	MSE	1.553e-03	1.054e-03	2.051e-03	2.080e-03

Table 9
Test results on the Lorenz chaotic time series dataset.

Dataset	Model	RMSE (95 % CI)	RMSE KBLCCG (Su & Wang, 2025)
Lorenz Chaotic	XGBoost	0.00453 (0.00422, 0.00484)	0.0686
Lorenz Chaotic	MLP	0.00779 (0.00726, 0.00833)	0.0686
Lorenz Chaotic	TPOT	0.00417 (0.00388, 0.00446)	0.0686

approach for improving performance on the Hénon dataset, whilst further exploration and research is needed to outperform the state-of-the-art method.

5.4.2. Lorenz chaotic time series

The second chaotic time series is derived from the Lorenz system.

Following Su and Wang (2025), we generated a series of 4000 points, and split the data into training, testing, and validation sets with a ratio of 2400:800:800.

Using the TPOT model, we could achieve a significant improvement in predictive performance compared to the baseline (Su & Wang, 2025). The results in Table 9 indicate that AutoML techniques like TPOT can effectively capture the complex dynamics of chaotic systems. This suggests promising potential for further application of such methods in time series forecasting and related domains.

5.4.3. Shanghai monthly and daily

In the following chapter, we present the results for the Shanghai dataset.

The final results for the Shanghai Monthly and Daily datasets are presented in Table 10 and Table 11, where we focused on MAE and RMSE, respectively. For the Shanghai Monthly dataset, the XGBoost model achieved an MAE of 448.34 ± 99.82 , a significant improvement over the baseline MAE of 675.95. This confirms that the optimization successfully enhanced the model accuracy.

On the other hand, for the Shanghai Daily dataset, XGBoost achieved an MAE of 20.94 with a 95 % CI ranging from 19.01 to 22.87. This result is only slightly better than the baseline MAE of 22.07, and the difference is not statistically significant. However, the optimized model still outperformed the baseline.

In summary, while the optimization produced a significant improvement in the Shanghai Monthly dataset, its effect on the Shanghai Daily dataset was more limited, with an insignificant improvement in accuracy.

5.4.4. TAIEX dataset

In the following section, we present the results for the TAIEX dataset.

The final model performance on the TAIEX dataset was evaluated using the RMSE and MAE metrics. Table 12 summarizes the top-performing models among XGBoost, TPOT, and MLP across the years 2010 to 2019.

Our pipeline outperformed the baseline model from the referenced paper (which relies on technical analysis) in terms of RSMSE in 8 out of 10 years. This result is noteworthy, as our approach did not incorporate technical analysis and instead leveraged the capabilities of AutoML. During those years, TPOT emerged as the best-performing model four times, while XGBoost achieved the top performance three times. These results highlight the effectiveness of our optimization strategy and demonstrate consistent improvement, particularly as we surpassed the baseline in absolute terms in nearly every year.

The MLP, optimized using neural architecture search, shows considerable potential. However, for the TAIEX dataset, it was the best model in only 2 out of 10 years for each RMSE, and in those cases, it only outperformed the baseline in absolute terms. This suggests that more testing is needed to identify the optimal use case (dataset) via the optimization strategy to fully benefit from the potential of this method.

Table 10
MAE results for Shanghai datasets.

Dataset	Model	MAE (95 % CI)	Baseline Model	Baseline MAE
Shanghai Monthly	XGB	448.34 (348.52, 548.16)	BPNN (Wang et al., 2011)	675.95
Shanghai Daily	XGB	20.94 (19.01, 22.87)	RC (Wang et al., 2021)	22.07

Table 11
RMSE results for Shanghai datasets.

Dataset	Model	RMSE (95 % CI)	Baseline model	Baseline RMSE
Shanghai Monthly	MLP	510.90 (353.65, 668.15)	BPNN (Wang et al., 2011)	847.58
Shanghai Daily	TPOT	29.22 (26.48, 31.95)	RC (Wang et al., 2021)	30.76

Table 12
Test results for TAIEX datasets.

Model	Year	RMSE (95 % CI)	MAE (95 % CI)	FTS-AP-SVR (Wu et al., 2021)	
				RMSE	MAE
XGB	2010	54.8 (39.8, 69.9)	43.2 (39.8, 69.9)	83.99	66.07
TPOT	2011	145.0 (110.6, 179.3)	123.1 (110.6, 179.3)	143.37	112.32
MLP	2012	63.7 (46.1, 81.3)	45.9 (46.1, 81.3)	66.76	51.14
XGB	2013	48.7 (34.6, 62.7)	40.1 (34.6, 62.7)	61.65	49.39
TPOT	2014	78.0 (58.0, 98.0)	64.3 (58.0, 98.0)	85.09	69.04
XGB	2015	79.6 (56.8, 102.3)	65.3 (56.8, 102.3)	91.29	75.6
MLP	2016	83.5 (62.7, 104.3)	62.3 (62.7, 104.3)	91.31	67.62
TPOT	2017	58.4 (41.6, 75.1)	47.3 (41.6, 75.1)	75.57	60.91
TPOT	2018	112.4 (82.3, 142.5)	85.8 (82.3, 142.5)	88.91	67.3
TPOT	2019	71.2 (50.4, 91.9)	57.8 (50.4, 91.9)	101.62	81.08

Table 13
Test results for the smaller TAIEX dataset (Nov-Dec 2014).

Model	Date range	RMSE (95 % CI)	MAE (95 % CI)	PIFS-ARC-Markov (Kumar & Pant, 2025)	
				RMSE	MAE
MLP	Nov-Dec 2014	29.91 (11.46, 48.36)	22.50 (10.28, 34.72)	30.80	21.27

Table 14
Prediction performance comparison: GJO-GRU vs. proposed approach.

Ticker	GJO-GRU LSTM (Gkonis & Tsakalos, 2025)		Best model	Proposed	
	RMSE	MAE		RMSE	MAE
AAPL	2.652	1.943	XGBoost	2.528	2.058
AMZN	3.040	2.316	XGBoost	2.747	2.114
AVGO	3.750	2.526	XGBoost	2.862	1.984
BRK-B	3.432	2.728	XGBoost	2.945	2.339
COST	14.301	11.628	TPOT	7.569	5.218
GOOGL	3.292	2.546	XGBoost	2.553	1.700
JPM	3.088	2.289	XGBoost	2.120	1.619

For the TAIEX dataset, we consistently outperformed the baseline in absolute RMSE terms nearly every year, underscoring the importance of optimizing the fuzzy pipeline through the tuning process. Additionally, we calculated the 95 % confidence interval for our predictions and achieved significant improvement in 3 out of 10 years.

Additionally, we extended our evaluation by comparing our method against results from a recent study by Kumar and Pant (2025), which used a single, smaller-sized dataset described in Table. This comparison allows us to benchmark our model against current literature. Results of this experiment are reported in Table 13.

Our method outperforms the baseline in terms of RMSE. However, due to the limited size of the dataset, statistical significance cannot be established. Nonetheless, this result provides further evidence of the com-

petitiveness of our approach when compared to recent state-of-the-art models.

For the sake of completeness, we also report results from other studies on the TAIEX dataset. Yolcu and Lam (2017) benchmarked their method against 10 different baseline models, achieving an average performance of 70.79 across the years 2010-2016. While a direct comparison with our results is not feasible due to differences in the number of samples and train/test splits, their findings provide useful insights and may serve as a foundation for future testing and comparisons in extending this study. On the other hand, Kocak et al. (2020) evaluated their NN model using data from a two-month range in 1998, with only 31 samples for training and 16 for testing, reporting an RMSE of 65.30. Given their choice of year and the limited dataset size, a direct comparison is not possible. Furthermore, larger sample sizes are necessary to fully leverage the capabilities of AutoML solutions, as demonstrated in our study spanning the years 2010-2019, where each year provided at least 220 training samples.

5.4.5. Selected stocks from S&P 500 index

To ensure the relevance of our approach to recent data and real-world forecasting challenges, we additionally considered the stock prices of seven major companies from the S&P 500 index, spanning diverse industries such as technology and finance. This inclusion further enhances the diversity of our datasets and demonstrates the generalizability of our model to various data.

In Table 14, we present key metrics of our final prediction model, along with the performance of the associated classification component,

Table 15
Univariate forecasting test results across currencies (RMSE and MAE).

Year	Currency pair	Baseline method	Baseline RMSE	Proposed method	
				RMSE	MAE
2015	USD/TRY	FF-ANN-PSO (Egrioglu et al., 2016)	0.0266	0.0229	0.0186
2016	USD/TRY	MPFIS-PSANN with GBP (Bas et al., 2023)	0.3259	0.0412	0.0329
2017	USD/TRY	MPFIS-PSANN with EUR (Bas et al., 2023)	0.0658	0.0297	0.0226
2018	USD/TRY	TSFIS (Yolcu & Alpaslan, 2018)	0.1218	0.0468	0.0339
2015	EUR/TRY	MPFIS-PSANN with GBP (Bas et al., 2023)	0.0354	0.0411	0.0363
2016	EUR/TRY	TSFIS (Yolcu & Alpaslan, 2018)	0.1955	0.0330	0.0253
2017	EUR/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1721	0.0347	0.0268
2018	EUR/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1237	0.0507	0.0378
2015	GBP/TRY	MPFIS-PSANN with EUR (Bas et al., 2023)	0.0508	0.0543	0.0465
2016	GBP/TRY	FF-ANN-PSO (Egrioglu et al., 2016)	0.0745	0.0487	0.0375
2017	GBP/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1589	0.0398	0.0303
2018	GBP/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1108	0.1208	0.0835

Table 16
Multivariate forecasting results across currencies (RMSE and MAE).

Year	Currency pair	Baseline method	Baseline RMSE	Proposed method	
				RMSE	MAE
2015	USD/TRY	FF-ANN-PSO (Egrioglu et al., 2016)	0.0266	0.0229	0.0192
2016	USD/TRY	MPFIS-PSANN with GBP (Bas et al., 2023)	0.3259	0.0488	0.0418
2017	USD/TRY	MPFIS-PSANN with EUR (Bas et al., 2023)	0.0658	0.0298	0.0222
2018	USD/TRY	TSFIS (Yolcu & Alpaslan, 2018)	0.1218	0.0568	0.0404
2015	EUR/TRY	MPFIS-PSANN with GBP (Bas et al., 2023)	0.0354	0.0419	0.0357
2016	EUR/TRY	TSFIS (Yolcu & Alpaslan, 2018)	0.1955	0.0401	0.0314
2017	EUR/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1721	0.0335	0.0250
2018	EUR/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1237	0.0512	0.0387
2015	GBP/TRY	MPFIS-PSANN with EUR (Bas et al., 2023)	0.0508	0.0377	0.0311
2016	GBP/TRY	FF-ANN-PSO (Egrioglu et al., 2016)	0.0745	0.0487	0.0388
2017	GBP/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1589	0.0444	0.0346
2018	GBP/TRY	MPFIS-PSANN with USD (Bas et al., 2023)	0.1108	0.1079	0.0772

and compare the results to the baseline GJO-GRU|LSTM (Gkonis & Tsakalos, 2025). XGBoost is reported as the best model in case of equal performance due to its faster execution time.

5.4.6. Turkish Lira exchange rates datasets

To increase the diversity of our datasets, we also included the exchange rate data, focusing on three major foreign currencies relative to the Turkish Lira. We then compared our results to the MPFIS-PSANN proposed by Bas et al. (2023), to the TS-FIS discussed by Yolcu and Alpaslan (2018), and FF-ANN-PSO presented by Egrioglu et al. (2016), using the same currencies. Results of univariate forecasting are summarized in Table 15.

By employing univariate forecasting, we were able to outperform all yearly datasets for USD, three out of four for EUR, and two for GBP. These results highlight the effectiveness of this approach when applied to exchange rate data.

To move closer to addressing real-world forecasting challenges, it is essential to incorporate external variables rather than relying solely on historical price data. To this end, we employed our model to support multivariate forecasting by integrating additional relevant features. The obtained results are collected in Table 16.

Multivariate forecasting showed mixed success. Its most notable advantage was observed with GBP, where it outperformed the baseline in 3 out of 4 years, surpassing even the univariate approach.

6. Conclusion and future work

In this paper, we proposed a robust pipeline combining fuzzy logic and AutoML, leveraging advanced optimization techniques to efficiently identify optimal configurations and fine-tune predictive models for superior performance. The experiments were mainly conducted in the application field of stock price forecasting, considering both univariate and multivariate cases.

The results and methodology presented in this study introduce several significant advancements in the domain of FTS, particularly within the context of stock price forecasting. By reframing the forecasting task as a classification problem rather than a regression one, we address a critical limitation in classical FTS models, i.e. the assumption of a fixed MF structure. This shift allows for greater flexibility in handling uncertainty and enhances compatibility with modern machine learning paradigms.

The use of AutoML to optimize fuzzy partitioning represents, to the best of our knowledge, a novel contribution to the FTS literature. This framework not only automates the selection of the number and type of fuzzy sets but also identifies the optimal number of time lags, thereby eliminating the need for extensive manual tuning and expert-driven heuristics. This is a crucial development in enabling the widespread and scalable application of FTS models in financial forecasting.

A key strength of the proposed approach lies in its preservation of interpretability, a quality often compromised in hybrid models that combine fuzzy systems with machine learning. Unlike many recent methods that obscure the internal decision process through deep or opaque architectures, our approach maintains a clear mapping from fuzzified inputs to decision outcomes. The use of logical aggregation at the inference stage, as opposed to learned black-box mechanisms, ensures that the reasoning behind forecasts can be traced and understood.

This interpretability is of particular importance to decision-makers, such as financial analysts, investors, and policy-makers, who often require transparent models that justify their predictions. In high-stakes domains like finance, where trust and accountability are critical, models that provide insight into the reasoning behind a forecast are far more actionable than those offering only raw predictive accuracy. By retaining a strong connection to fuzzy set theory and leveraging interpretable features (e.g., membership values and fuzzy set positioning), our model supports informed and explainable decision-making.

It must also be mentioned that the proposed methodology has applications beyond finance. Domains characterized by vague or imprecise data, such as healthcare, energy demand forecasting, and environmental modelling, could benefit from this AutoML-driven fuzzy framework. Exploring such application could be part of future work.

Anyhow, several directions for future research emerge from this work. The procedure could be adapted to any fuzzy system, as the fuzzy partitioning is the first common stage. This could lead to dynamic and adaptive fuzzy systems. More precisely, future models may benefit from incorporating mechanisms for adaptive fuzzy partitioning, where membership functions evolve in real-time based on incoming data, better capturing the non-stationarity inherent in some fields besides financial markets. Further exploration could involve deep learning models, such as recurrent neural networks, to lock improved long-term dependency modelling while retaining fuzzy-based interpretability. Another interesting aspect would be introducing formal interpretability metrics. Although our method prioritizes interpretability by design, there is a need for quantitative evaluation of interpretability. Future work should aim to establish formal metrics or frameworks to assess and compare the transparency and comprehensibility of FTS models, particularly in hybrid settings. Last but not least, it must be mentioned that despite the variability in performance, multivariate forecasting remains an important extension, as incorporating external features is likely essential for capturing more complex patterns. Further development and exploration in this direction will also be part of future work.

In conclusion, this work presents a flexible, automated, and interpretable framework for fuzzy time series forecasting. It not only improves forecasting accuracy but also ensures that the resulting models remain transparent and usable for real-world decision support, a critical requirement in domains where understanding the rationale behind predictions is just as important as the predictions themselves.

CRediT authorship contribution statement

Jan Timko: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing – original draft, Writing - review & editing; **Radwa El Shawi:** Conceptualization, Methodology, Resources, Supervision, Writing – original draft, Writing – review & editing; **Stefania Tomasiello:** Conceptualization, Methodology, Formal analysis, Resources, Supervision, Writing – original draft, Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Abd Elrahman, A., El Helw, M., Elshawi, R., & Sakr, S. (2020). D-smartML: A distributed automated machine learning framework. In *2020 IEEE 40th international conference on distributed computing systems (ICDCS)* (pp. 1215–1218). IEEE.
- Amini, A., Abdollahi, A., & Hariri-Ardebili, M. A. (2024). An automated machine-learning-assisted stochastic-fuzzy multi-criteria decision making tool: Addressing record-to-record variability in seismic design. *Applied Soft Computing*, 154, 111354.
- Bai, E., Wong, W. K., Chu, W., Xia, M., & Pan, F. (2011). A heuristic time-invariant model for fuzzy time series forecasting. *Expert Systems with Applications*, 38(3), 2701–2707.
- Bartlett, P., Jordan, M., & McAuliffe, J. (2006). Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 101, 138–156.
- Bas, E., Egrioglu, E., & Tunc, T. (2023). Multivariate picture fuzzy time series: New definitions and a new forecasting method based on pi-sigma artificial neural network. *Computational Economics*, 61, 139–164. Published online: 29 September 2021 <https://doi.org/10.1007/s10614-021-10202-w>
- Batrouti, M., (2020). A hybrid architecture for tiered storage with fuzzy logic and automl. In Y. Luo, *Proceedings of the international conference on Cooperative Design, Visualization and Engineering (CDVE 2020)* (pp. 67–74). Springer, 12341, Lecture notes in computer science.
- Bishop, C. M. (1995). Neural networks for pattern recognition. Oxford University Press.
- Bisht, K., & Kumar, A. (2023). A method for fuzzy time series forecasting based on interval index number and membership value using fuzzy c-means clustering. *Evolutionary Intelligence*, 16, 285–297.
- Bonissone, P. (1979). A pattern recognition approach to the problem of linguistic approximation in system analysis. In *IEEE international conference on cybernetics and society* (pp. 793–798).
- Box, G. E. P. (2015). Time series analysis: Forecasting and control. Wiley.
- Chen, S. M. (1996). Forecasting enrollments based on fuzzy time series. *Fuzzy Sets and Systems*, 81(3), 311–319.
- Chen, S. M. (2002). Forecasting enrollments based on high-order fuzzy time series. *Cybernetics and Systems*, 33(1), 1–16.
- Chen, S.-M., Zou, X.-Y., & Gunawan, G. C. (2019). Fuzzy time series forecasting based on proportions of intervals and particle swarm optimization techniques. *Information Sciences*, 500, 127–139.
- Cheng, C.-H., Cheng, G.-W., & Wang, J. W. (2008). Multi-attribute fuzzy time series method based on fuzzy clustering. *Expert Systems with Applications*, 34(2), 1235–1242.
- DeGooijer, J. G., & Hyndman, R. J. (2006). 25 Years of time series forecasting. *International Journal of Forecasting*, 22(3), 443–473.
- Duru, O., & Bulut, E. (2014). A non-linear clustering method for fuzzy time series: histogram damping partition under the optimized cluster paradox. *Applied Soft Computing*, 24, 742–748.
- Egrioglu, E., Aladag, C. H., & Yolcu, U., et al. (2010). Finding an optimal interval length in high order fuzzy time series. *Expert Systems with Applications*, 37, 5052–5055.
- Egrioglu, E., Bas, E., Aladag, C. H., & Yolcu, U. (2016). Probabilistic fuzzy time series method based on artificial neural network. *American Journal of Intelligent Systems*, 6(2), 42–47.
- El Shawi, R., Bahman, M., & Sakr, S. (2025). To tune or not to tune? an approach for recommending important hyperparameters for classification and clustering algorithms. *Future Generation Computer Systems*, 163, 107524.
- Eldeeb, H., Amashukeli, S., & ElShawi, R. (2022). Bigfeat: Scalable and interpretable automated feature engineering framework. In *2022 IEEE international conference on big data (big data)* (pp. 515–524). IEEE.
- Eldeeb, H., Maher, M., Elshawi, R., & Sakr, S. (2024). AutoMLBench: A comprehensive experimental evaluation of automated machine learning frameworks. *Expert Systems with Applications*, 243, 122877.
- Feurer, M., Klein, A., Eggensperger, K., Springenberg, J., Blum, M., & Hutter, F. (2015). Efficient and robust automated machine learning. In *Advances in neural information processing systems*. (vol. 28).
- García, S., Luengo, J., Saez, J. A., Lopez, V., & Herrera, F. (2013). A survey of discretization techniques: Taxonomy and empirical analysis in supervised learning. *IEEE Transactions on Knowledge and Data Engineering*, 25(4).
- Gautam, S. S., & Abhishek (2019). A novel moving average forecasting approach using fuzzy time series data set. *Journal of Control, Automation and Electrical Systems*, 30, 532–544.
- Gkonis, V., & Tsakalos, I. (2025). A hybrid optimized deep learning model via the golden jackal optimizer for accurate stock price forecasting. *Expert Systems with Applications*, 278, 127287. Open access under a Creative Commons license <https://doi.org/10.1016/j.eswa.2025.127287>
- Goyal, G., Dinesh, G., & Bisht, C. S. (2023). Adaptive hybrid fuzzy time series forecasting technique based on particle swarm optimization. *Granular Computing*, 8, 373–390.
- Huang, K. (2001). Effective lengths of intervals to improve forecasting in fuzzy time series. *Fuzzy Sets and Systems*, 123, 387–394.
- Huang, K., & Yu, T. H. K. (2006). Ratio-based lengths of intervals to improve fuzzy time series forecasting. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 36, 328–340.
- Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2011). Sequential model-based optimization for general algorithm configuration. In *Learning and intelligent optimization: 5th international conference, LION 5, Rome, Italy, January 17–21, 2011. selected papers*. Springer (vol. 5).
- Hutter, F., Kotthoff, L., & Vanschoren, J. (2019). Automated machine learning: Methods, systems, challenges. Springer Nature.
- Hwang, J.-R., Chen, S.-M., & Lee, C. H. (1998). Handling forecasting problems using fuzzy time series. *Fuzzy Sets and Systems*, 100(1–3), 217–228.
- Jayawardena, A. W., Xu, P. C., Tsang, F. L., & Li, W. K. (2006). Determining the structure of a radial basis function network for prediction of nonlinear hydrological time series. *Hydrological Sciences Journal*, 51(1), 21–44. <https://doi.org/10.1623/hysj.51.1.21>
- Jiang, P., Dong, Q., Li, P., & Lian, L. (2017). A novel high-order weighted fuzzy time series model and its application in nonlinear time series prediction. *Applied Soft Computing*, 55, 44–62.
- Kantz, H., & Schreiber, T. (2003). Nonlinear time series analysis. (2nd ed.). Cambridge University Press.
- Kocak, C., Dalar, A. Z., Yolcu, O. C., Bas, E., & Egrioglu, E. (2020). A new fuzzy time series method based on an ARMA-type recurrent pi-sigma artificial neural network. *Soft Computing*, 24, 8243–8252.
- Kumar, S., & Pant, S. (2025). Pifs, arc and markov model based hybrid method for fuzzy time series forecasting. *Expert Systems with Applications*, 279(3), 127510. <https://doi.org/10.1016/j.eswa.2025.127510>

- Kuo, I.-H., Horng, S.-J., Kao, T.-W., Lin, T.-L., Lee, C.-L., & Pan, Y. (2009). An improved method for forecasting enrollments based on fuzzy time series and particle swarm optimization. *Expert Systems with Applications*, 36(3), 6108–6117.
- LeDell, E., & Poirier, S. (2020). H₂O AutoML: Scalable automatic machine learning. In *Proceedings of the 7th ICML workshop on automated machine learning (autoML 2020)*. ICML.
- Liu, Z. (2023). Forecasting stock prices based on multivariable fuzzy time series. *AIMS Mathematics*, 8(6), 12778–12792.
- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2), 130–141.
- Nayak, J., Naik, B., Dash, P. B., & Pelusi, D. (2021). Optimal fuzzy cluster partitioning by crow search meta-heuristic for biomedical data analysis. *International Journal of Applied Metaheuristic Computing*, 12(2), 49–66.
- Olson, R. S., Urbanowicz, R. J., Andrews, P. C., Lavender, N. A., La Creis, K., & Moore, J. H. (2016). Applications of evolutionary computation: 19th European conference, evoapplications 2016, Porto, Portugal, March 30, – April 1, 2016, proceedings, part I. (pp. 123–137). Springer International Publishing. https://doi.org/10.1007/978-3-319-31204-0_9
- Ott, E., Grebogi, C., & Yorke, J. A. (1990). Controlling chaos. *Physical Review Letters*, 64(11), 1196–1199.
- Panigrahi, S., & Behera, H. S. (2020). A study on leading machine learning techniques for high-order fuzzy time series forecasting. *Engineering Applications of Artificial Intelligence*, 87, 103245.
- Park, M., Lee, M. L., & Lee, J. (2019). Predicting stock market indices using classification tools. *Asian Economic and Financial Review*, 9(2), 243–256.
- PhamToan, D., VoThiHang, N., & PhamThi, B. (2024). Improving forecasting model for fuzzy time series using the self-updating clustering and bi-directional long short term memory algorithm. *Expert Systems with Applications*, 241, 122767.
- Rockafellar, R. T. (1997). *Convex analysis*. Princeton, NJ: Princeton University Press.
- Sadaei, H. J., de Lima e Silva, P. C., Guimaraes, F. G., & Lee, M. H. (2019). Short-term load forecasting by using a combined method of convolutional neural networks and fuzzy time series. *Energy*, 175, 365–377.
- Satriani, R. B. A., Farikhin, F., & Surarso, B. (2025). Stock price forecasting using fuzzy c-means and type-2 fuzzy time series. *BAREKENG: Jurnal Ilmu Matematika dan Terapan*, 19(2), 1365–1378.
- Silva Filho, T., Song, H., & Perello-Nieto, M., et al. (2023). Classifier calibration: A survey on how to assess and improve predicted class probabilities. *Machine Learning*, 112, 3211–3260.
- Singh, P. (2017). A brief review of modeling approaches based on fuzzy time series. *International Journal of Machine Learning & Cybernetics*, 8, 397–420.
- Singh, P. (2018). Rainfall and financial forecasting using fuzzy time series and neural networks based model. *International Journal of Machine Learning and Cybernetics*, 9, 491–506.
- Singh, P. (2020). A novel hybrid time series forecasting model based on neutrosophic-PSO approach. *International Journal of Machine Learning and Cybernetics*, 11(8), 1643–1658.
- Singh, S. (2007). A simple method of forecasting based on fuzzy time series. *Applied Mathematics and Computation*, 186(1), 330–339.
- Skarda, C. A., & Freeman, W. J. (1987). How brains make chaos in order to make sense of the world. *Behavioral and Brain Sciences*, 10(2), 161–173.
- Song, Q., & Chissom, B. S. (1993a). Forecasting enrollments with fuzzy time series-part i. *Fuzzy Sets and Systems*, 54(1), 1–9.
- Song, Q., & Chissom, B. S. (1993b). Fuzzy time series and its models. *Fuzzy Sets and Systems*, 54(3), 269–277.
- Su, L., & Wang, X. (2025). Kernel broad learning cauchy conjugate gradient algorithm for online chaotic time series prediction. *Neurocomputing*, 639, 130234. School of Science, Chongqing University of Technology, Chongqing 400054, China.
- Tealab, A., Hefny, H., & Badr, A. (2017). Forecasting of nonlinear time series using ANN. *Future Computing and Informatics Journal*, 2(1), 39–47.
- Tsiakmaki, M., Kostopoulos, G., Kotsiantis, S. & Ragos, O., (2021). Fuzzy-Based active learning for predicting student academic performance using auttml: A step-Wise approach. *Journal of Computing in Higher Education*. <https://doi.org/10.1007/s12528-021-09279-x>.
- Wang, J.-Z., Wang, J.-J., Zhang, G., & Guo, S. P. (2011). Forecasting stock indices with back propagation neural network. *Expert Systems with Applications*, 38, 14346–14355. <https://doi.org/10.1016/j.eswa.2011.04.222>
- Wang, P., Wang, K., Song, Y. et al. (2024). AutoLDT: A lightweight spatio-temporal decoupling transformer framework with autoML method for time series classification. *Scientific Reports*, 14, 29801. <https://doi.org/10.1038/s41598-024-36270-8>
- Wang, W.-J., Tang, Y., Xiong, J., & Zhang, Y. C. (2021). Stock market index prediction based on reservoir computing models. *Expert Systems with Applications*, 178, 115022. <https://doi.org/10.1016/j.eswa.2021.115022>
- Wong, W.-K., Bai, E., & Chu, A. W. C. (2010). Adaptive time-variant models for fuzzy-time series forecasting. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 40(6), 1531–1542.
- Wu, H., Long, H., Wang, Y., & Wang, Y. (2021). Stock index forecasting: A new fuzzy time series forecasting method. *Journal of Forecasting*, 40(4), 653–666. <https://doi.org/10.1002/for.2734>
- Yolcu, O. C., & Alpaslan, F. (2018). Prediction of TAIEX based on hybrid fuzzy time series model with single optimization process. *Applied Soft Computing*, 66, 18–33. <https://www.sciencedirect.com/science/article/pii/S1568494618300632>
- Yolcu, O. C., & Lam, H. K. (2017). A combined robust fuzzy time series method for prediction of time series. *Neurocomputing*, 247, 87–101. <https://doi.org/10.1016/j.neucom.2017.03.001>
- Zeng, S., Chen, S.-M., & Teng, M. O. (2019). Fuzzy forecasting based on linear combinations of independent variables, subtractive clustering algorithm, and artificial bee colony algorithm. *Information Sciences*, 484, 350–366.
- Zhan, T., He, Y., Deng, Y., & Li, Z. (2024). Differential convolutional fuzzy time series forecasting. *IEEE Transactions on Fuzzy Systems*, 32(3), 831–845.